

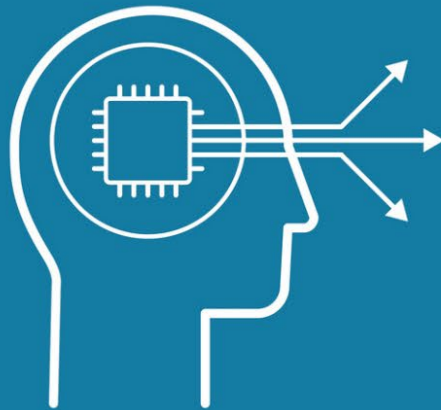
DEEP LEARNING

Ignacio Gómez
2025

1950

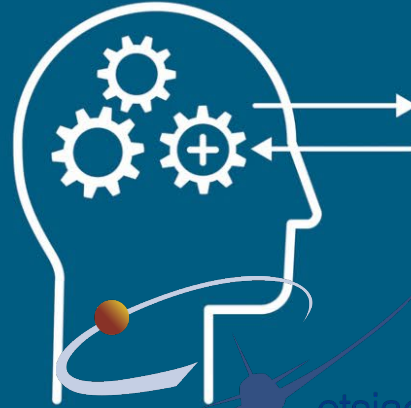
1980

2010



ARTIFICIAL INTELLIGENCE

ENGINEERING OF MACHINES
THAT MIMIC COGNITIVE FUNCTIONS



MACHINE LEARNING

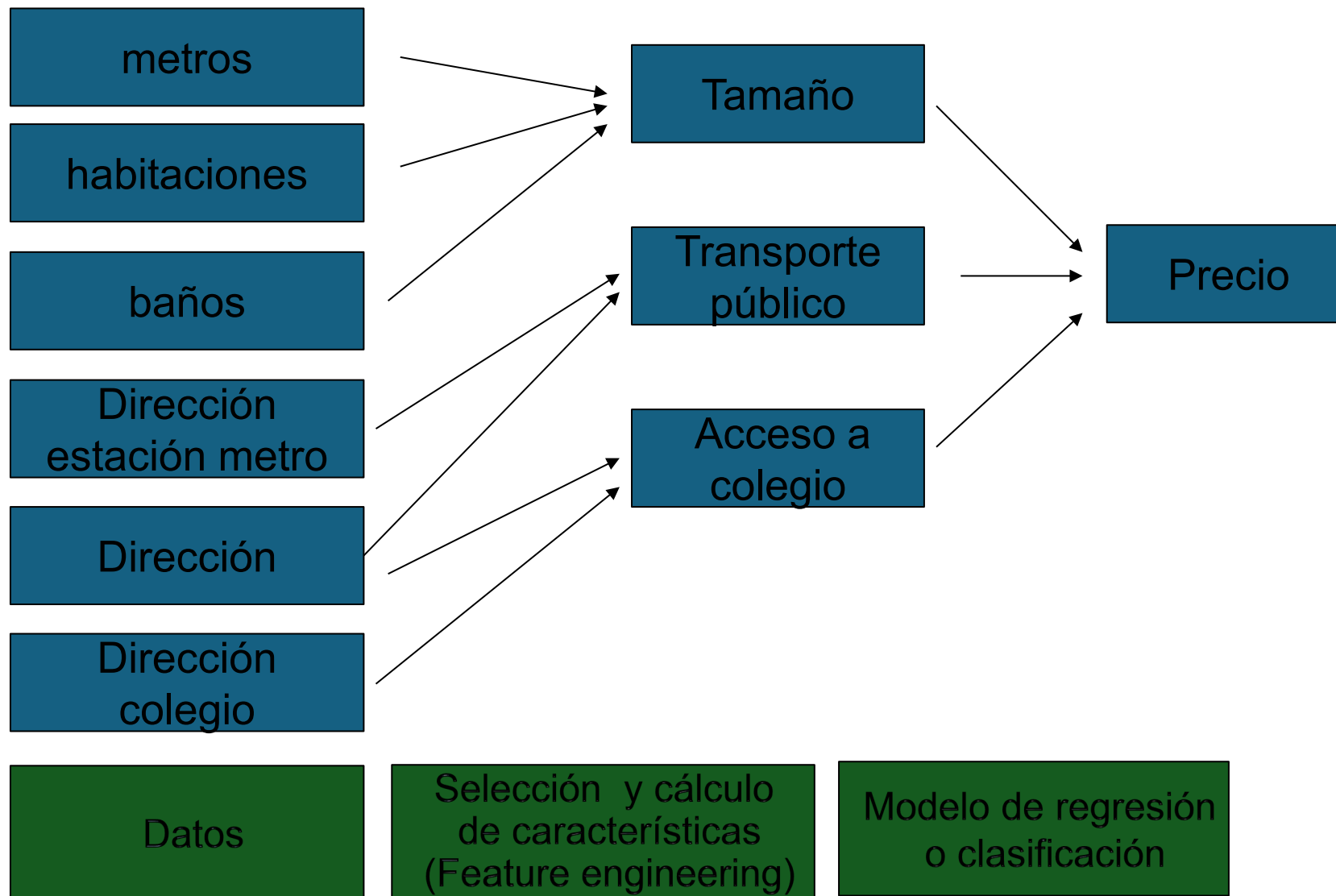
ABILITY TO PERFORM TASKS
WITHOUT EXPLICIT INSTRUCTIONS
AND RELYING ON PATTERNS

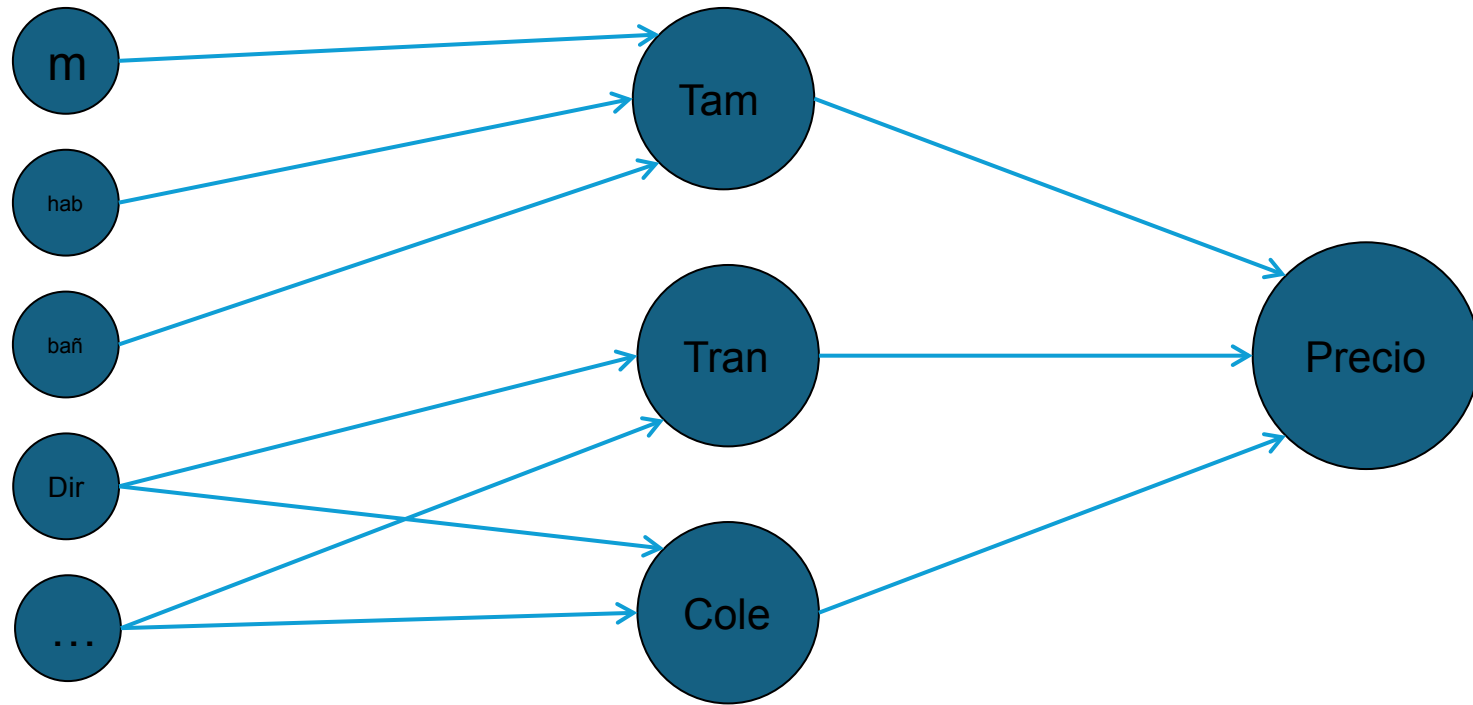
etsiae
UPM



DEEP LEARNING

MACHINE LEARNING BASED
ON ARTIFICIAL NEURAL NETWORKS

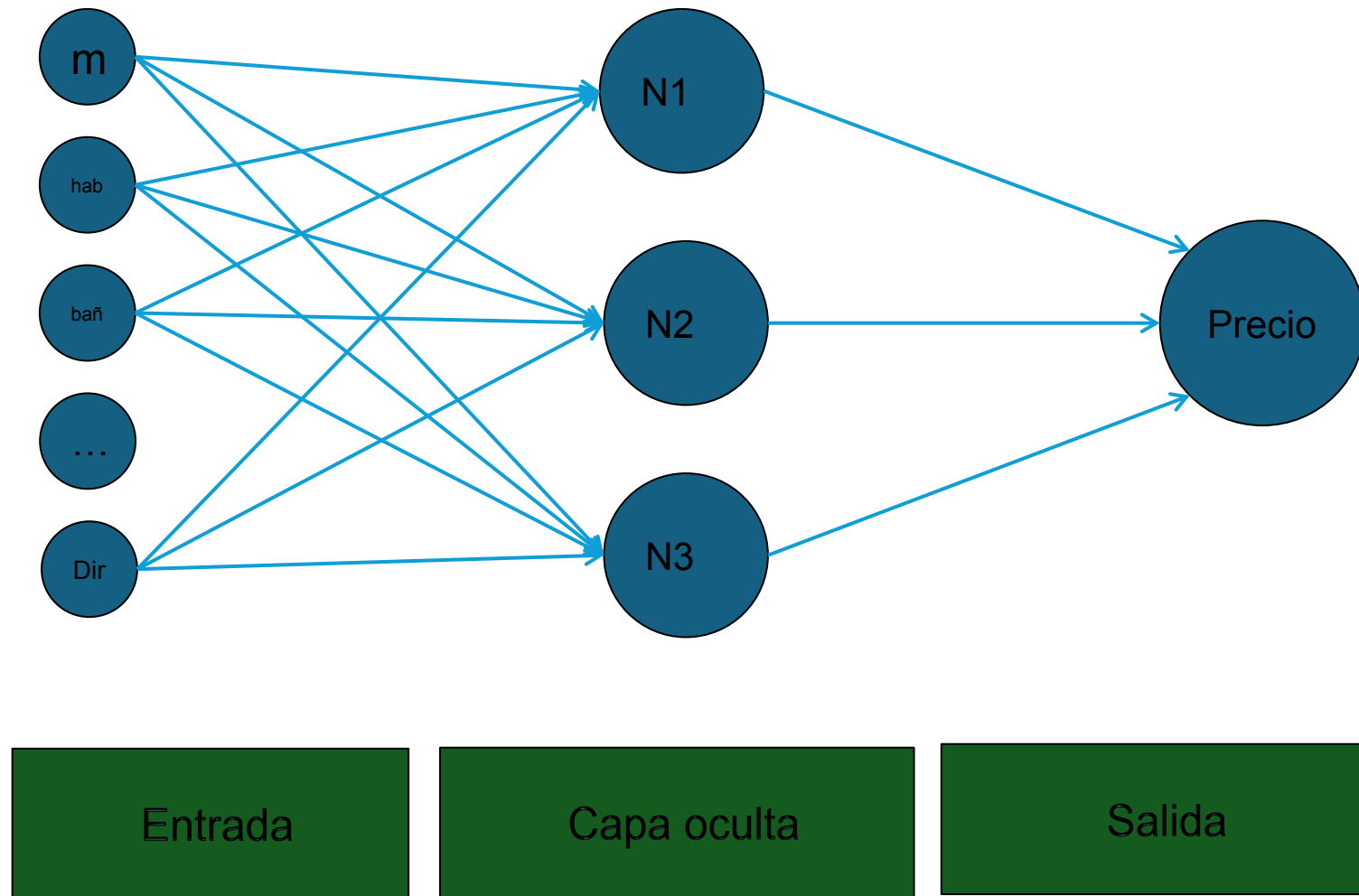


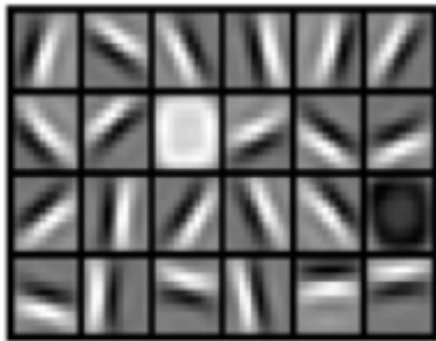
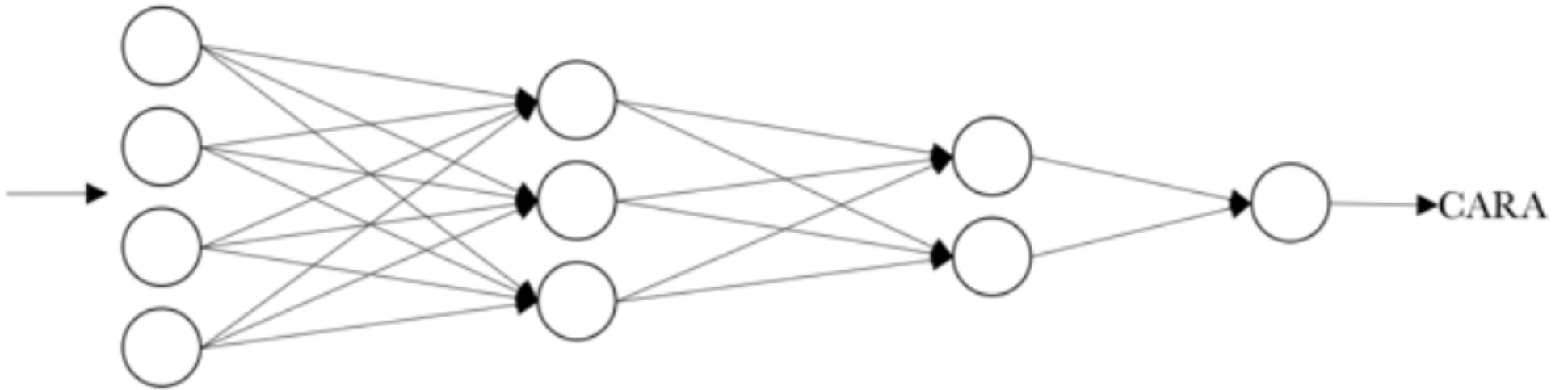


Datos

Selección y cálculo
de características
(Feature engineering)

Modelo de regresión
o clasificación





bordes

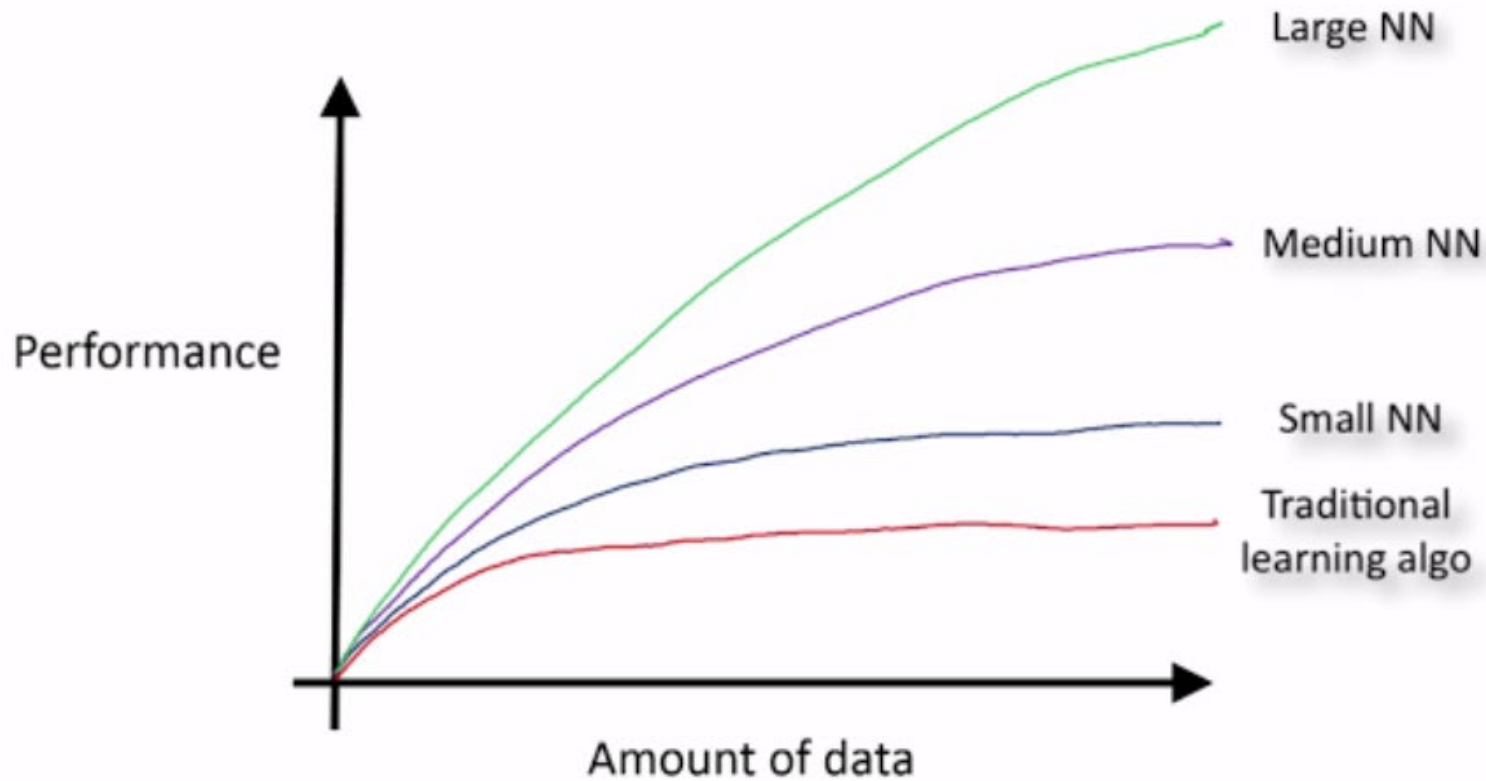


combinación de bordes



modelos de objetos

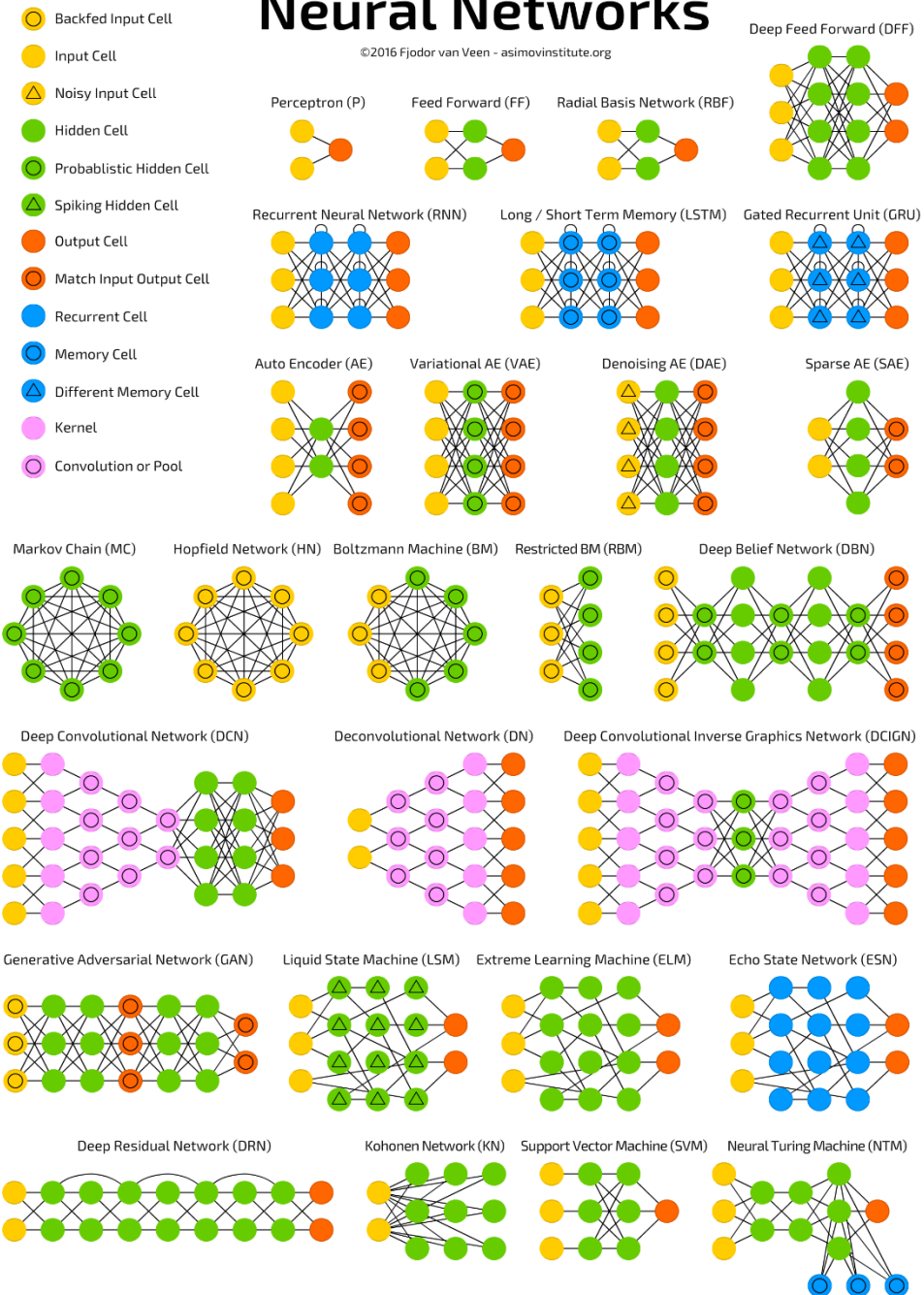
One picture explaining the rise of Deep Learning



Andrew Ng

Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org



- Redes Neuronales Feedforward: (FFNN o MLP)**

Las redes neuronales más básicas, donde la información fluye en una sola dirección: desde la capa de entrada, a través de capas ocultas (si las hay), hasta la capa de salida. No hay conexiones hacia atrás o dentro de la misma capa. Clasificación, regresión, reconocimiento de patrones.

- Redes Neuronales Profundas (Deep Neural Networks, DNN)**

Redes neuronales feedforward con múltiples capas ocultas (normalmente más de una o dos), lo que les permite aprender representaciones más complejas y abstractas de los datos. Reconocimiento de voz, procesamiento de imágenes, procesamiento del lenguaje natural.

- Redes Neuronales Recurrentes (Recurrent Neural Networks, RNN)**

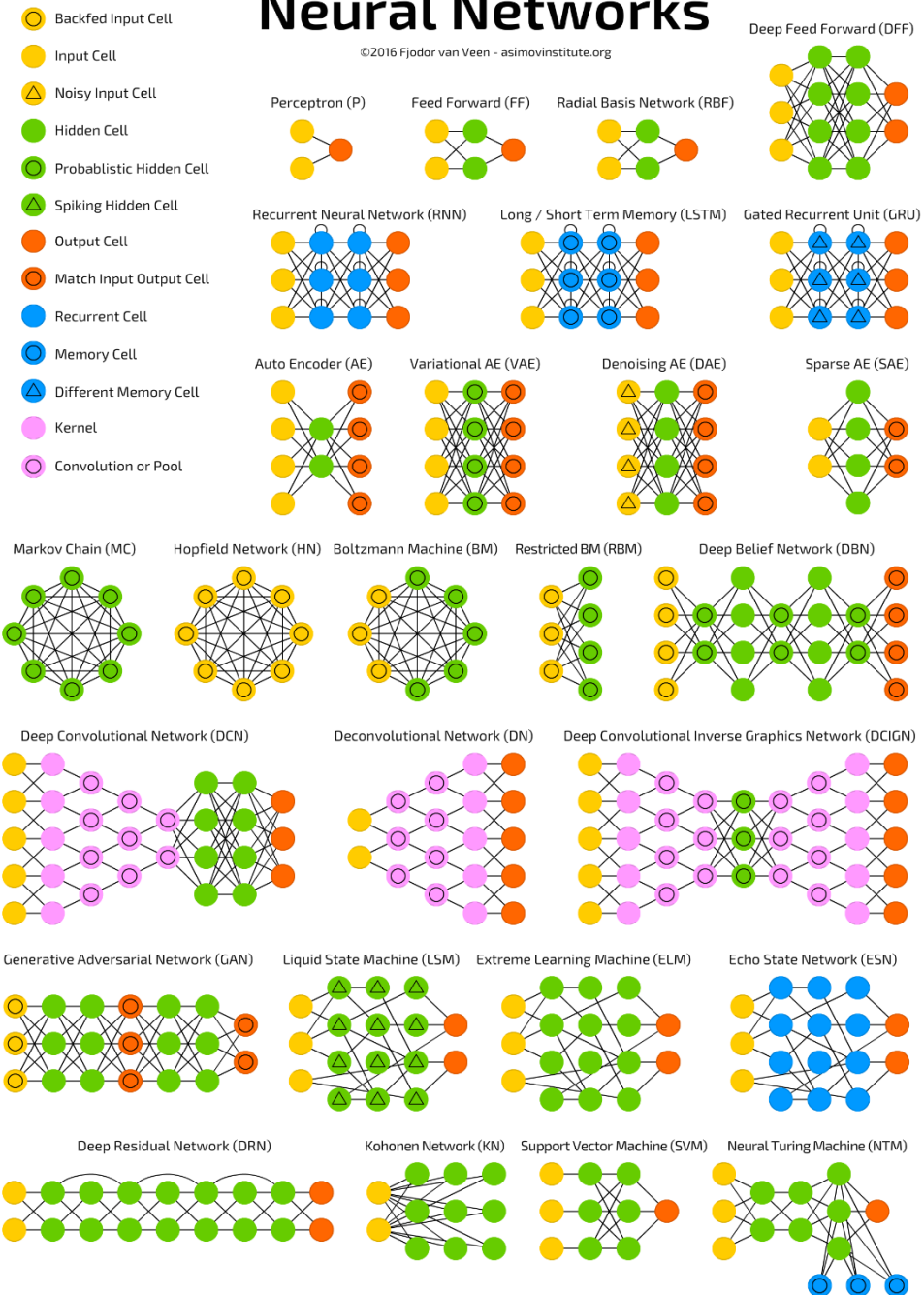
Las conexiones entre neuronas forman un ciclo, lo que permite que la información persista. Son ideales para datos secuenciales, ya que pueden tener memoria temporal de lo que han procesado anteriormente. Traducción automática, reconocimiento de voz, análisis de series temporales.

- Redes Neuronales Convolucionales (Convolutional Neural Networks, CNN)**

Diseñadas para procesar datos con una estructura de cuadrícula, como las imágenes. Usan capas de convolución para detectar patrones locales, como bordes o texturas, y capas de pooling para reducir dimensionalidad. Reconocimiento de imágenes, visión por computadora, análisis de video.

Neural Networks

©2016 Fjodor van Veen - asimovinstitute.org



- Redes Neuronales de Función de Base Radial (RBF)**
Utilizan funciones de base radial como funciones de activación. Se destacan por su capacidad de interpolación en problemas de clasificación, con una estructura que incluye una capa oculta que aplica una función de base radial y una capa de salida que realiza una combinación lineal de las salidas de la capa oculta. Reconocimiento de patrones, regresión no lineal, control adaptativo.

- Redes Neuronales Autoencoders**
Redes entrenadas para copiar su entrada en la salida. Tienen una estructura con una capa oculta (o más) más pequeña que la de entrada, forzando a la red a aprender una representación comprimida de los datos. Reducción de dimensionalidad, eliminación de ruido, generación de datos.

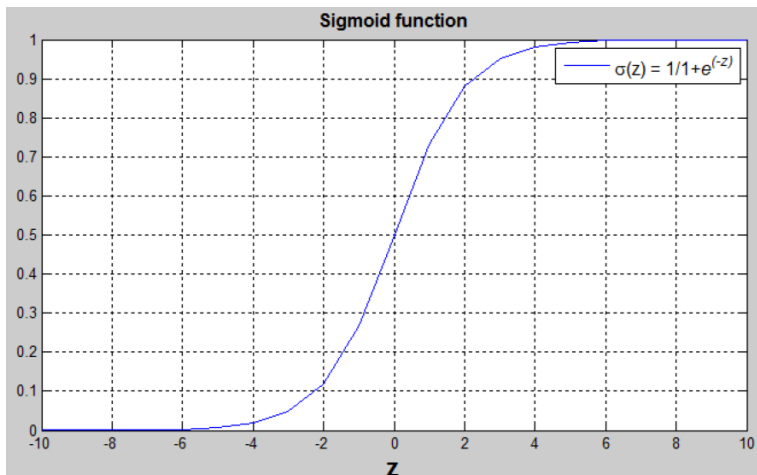
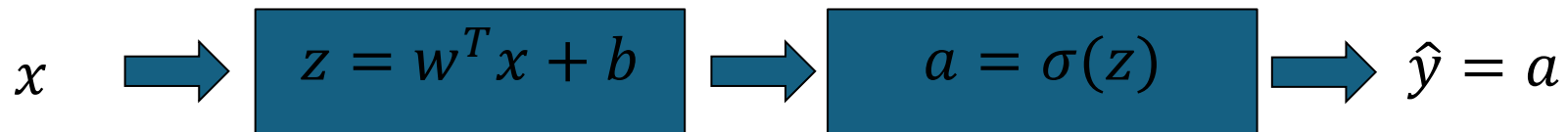
- Generative Adversarial Networks (GAN)**
Consisten en dos redes neuronales que compiten entre sí: un generador que crea datos falsos y un discriminador que intenta distinguir entre datos reales y falsos. El generador aprende a producir datos cada vez más realistas. Generación de imágenes y video, síntesis de voz, creación de obras de arte.

- Extreme Learning Machines (ELM)**
Variante de redes feedforward donde los pesos de las conexiones entre la capa de entrada y la capa oculta se asignan aleatoriamente y no se ajustan durante el entrenamiento. Solo se ajustan los pesos de la capa de salida, lo que hace que el entrenamiento sea extremadamente rápido. Clasificación, regresión, predicción de series temporales.

EL PERCEPTRON

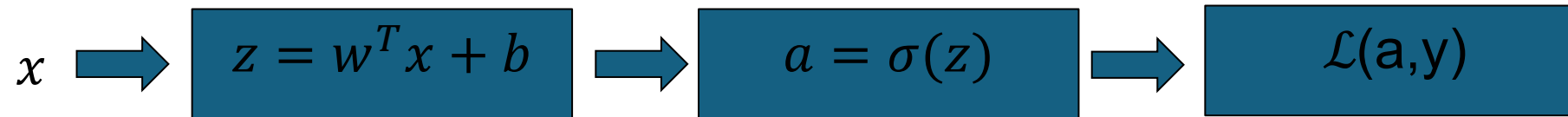
Ignacio Gómez
2025

Regresión Logística.



$$\mathcal{L}(a,y) = -(y \log(a) + (1-y)\log(1 - a)))$$

Forward calculation.



$$X = \begin{pmatrix} x_1^{(1)} & x_1^{(2)} & x_1^{(3)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & x_2^{(3)} & \dots & x_2^{(m)} \\ x_3^{(1)} & x_3^{(2)} & x_3^{(3)} & \dots & x_3^{(m)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ x_n^{(1)} & x_n^{(2)} & x_n^{(3)} & \dots & x_n^{(m)} \end{pmatrix}_{(n \times m)} \quad \text{where } X \in \mathbb{R}^{(n \times m)}$$

$$Z = W^T \cdot X + B$$

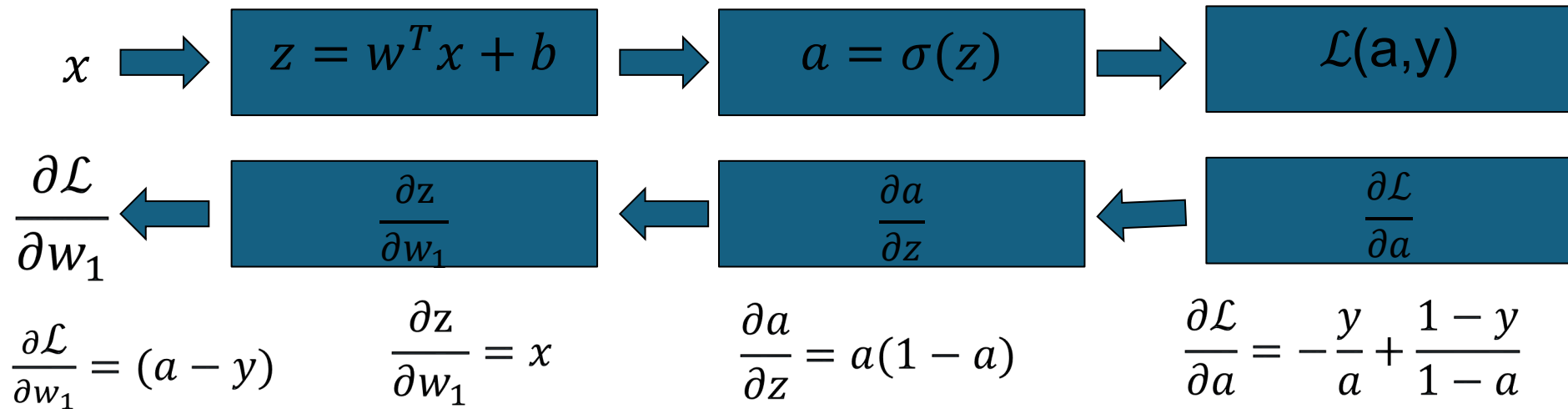
$$Y = (y^{(1)} \ y^{(2)} \ y^{(3)} \ \dots \ y^{(m)})_{(1 \times m)} \quad \text{where } y^{(i)} \in \{0, 1\}$$

$$A = \sigma(Z)$$

$$W = \begin{pmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{pmatrix}_{(n \times 1)} \quad \text{where } W \in \mathbb{R}^{(n \times 1)}$$

$$B = (b \ b \ b \ \dots \ b)_{(1 \times m)} \quad \text{where } b \in \mathbb{R}$$

Derivadas de la función de costes



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$\frac{\partial L}{\partial w} = (a - y)x$$

$$\frac{1}{m} X(A - Y)^T$$

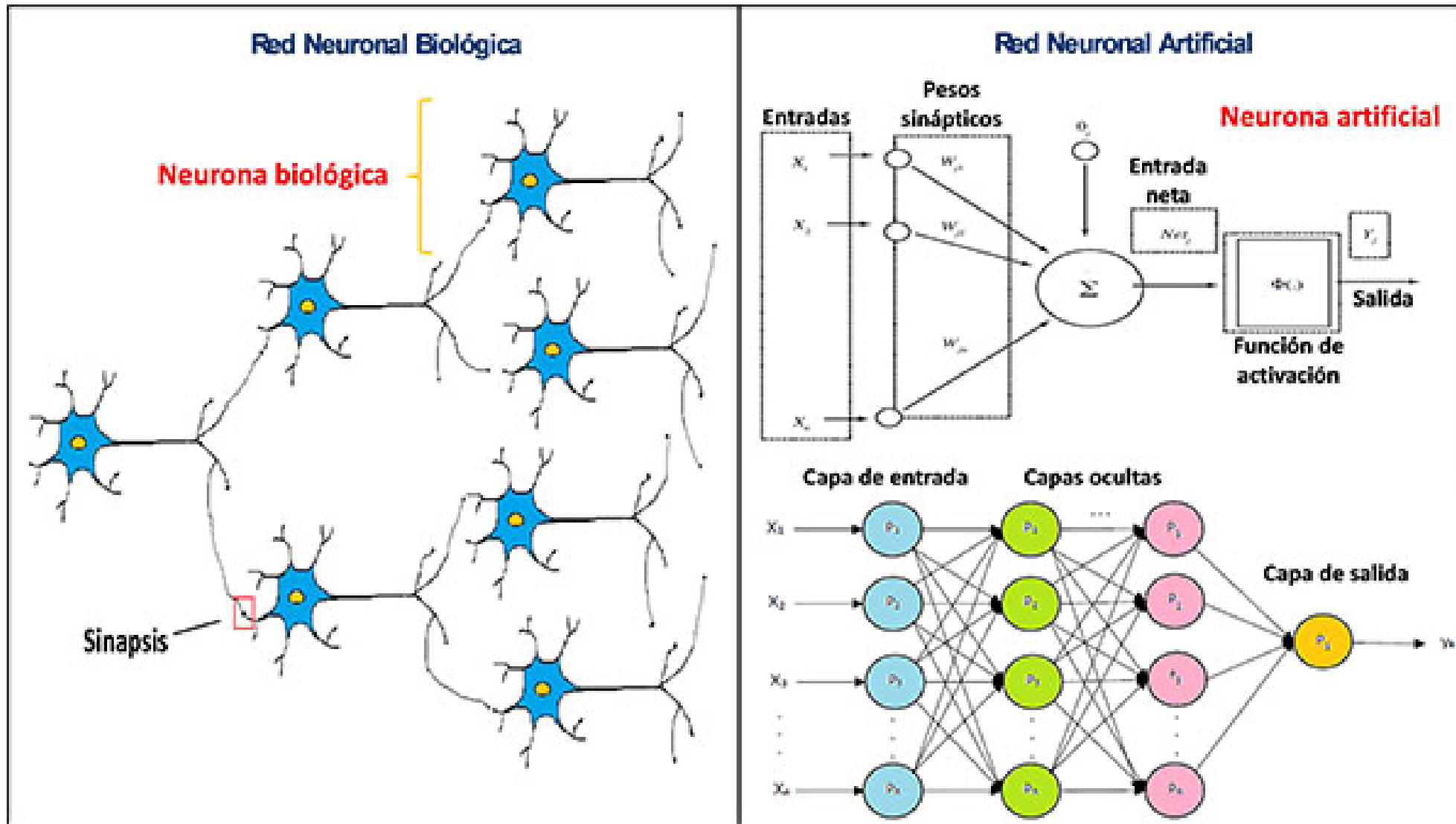
```
A = sigmoid(np.dot(w.T, X) + b) # compute activation
cost = -np.sum(Y * np.log(A) + (1 - Y) * np.log(1 - A)) / m #cost

dw = np.dot(X, (A - Y).T) / m
db = np.sum(A - Y) / m

# optimize
w = w - learning_rate * dw
b = b - learning_rate * db
```

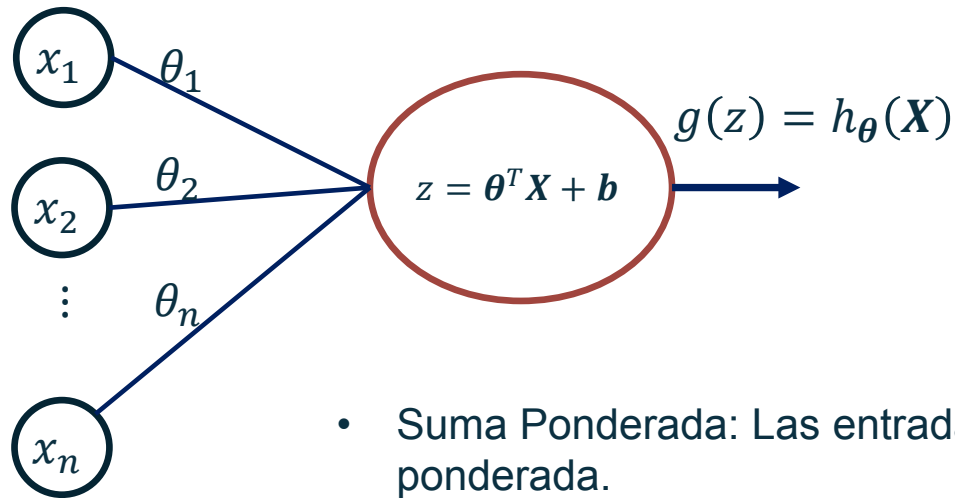
Regresion Logistica como intro a las ANN.ipynb

Redes Neuronales Artificiales



Perceptrón.

Inventado por **Frank Rosenblatt** en 1957. Fue uno de los primeros modelos de redes neuronales artificiales y ganó mucha atención en su momento, pero sus limitaciones llevaron a un declive en su popularidad en los años 70



- Input: $\mathbf{X} = \begin{Bmatrix} x_1 \\ \vdots \\ x_n \end{Bmatrix}$ Representan los datos de entrada a la neurona.
- Pesos: $\boldsymbol{\theta} = \begin{Bmatrix} b \\ \theta_1 \\ \vdots \\ \theta_n \end{Bmatrix}$ Cada entrada está multiplicada por un peso que determina la importancia de la entrada en la decisión final del perceptrón.

- Suma Ponderada: Las entradas multiplicadas por sus respectivos pesos se suman para formar una suma ponderada.

$$z = \boldsymbol{\theta}^T \mathbf{X} + \mathbf{b} = b + \theta_1 x_1 + \cdots + \theta_n x_n$$

- Función de Activación: La suma ponderada pasa a través de una función de activación que decide si la neurona "dispara" o no.

$$g(z) = \begin{cases} 1 \\ 0 \end{cases} \quad (\text{Más adelante veremos que no es del todo así})$$

Perceptrón: Mecanismo de actualización

- **1. La Predicción**

Calculamos la combinación lineal y aplicamos la función escalón:

$$z = w^T x + b$$
$$\hat{y} = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$$

- **2. El Cálculo del Error**

Comparamos la etiqueta real (y) con nuestra predicción ($\hat{y}$):

$$\text{Error} = y - \hat{y}$$

- **3. La Regla de Actualización**

Ajustamos los parámetros usando el error y la tasa de aprendizaje (α):

$$w_{nuevo} = w_{actual} + \alpha(y - \hat{y})x$$
$$b_{nuevo} = b_{actual} + \alpha(y - \hat{y})$$

Historia de las redes neuronales

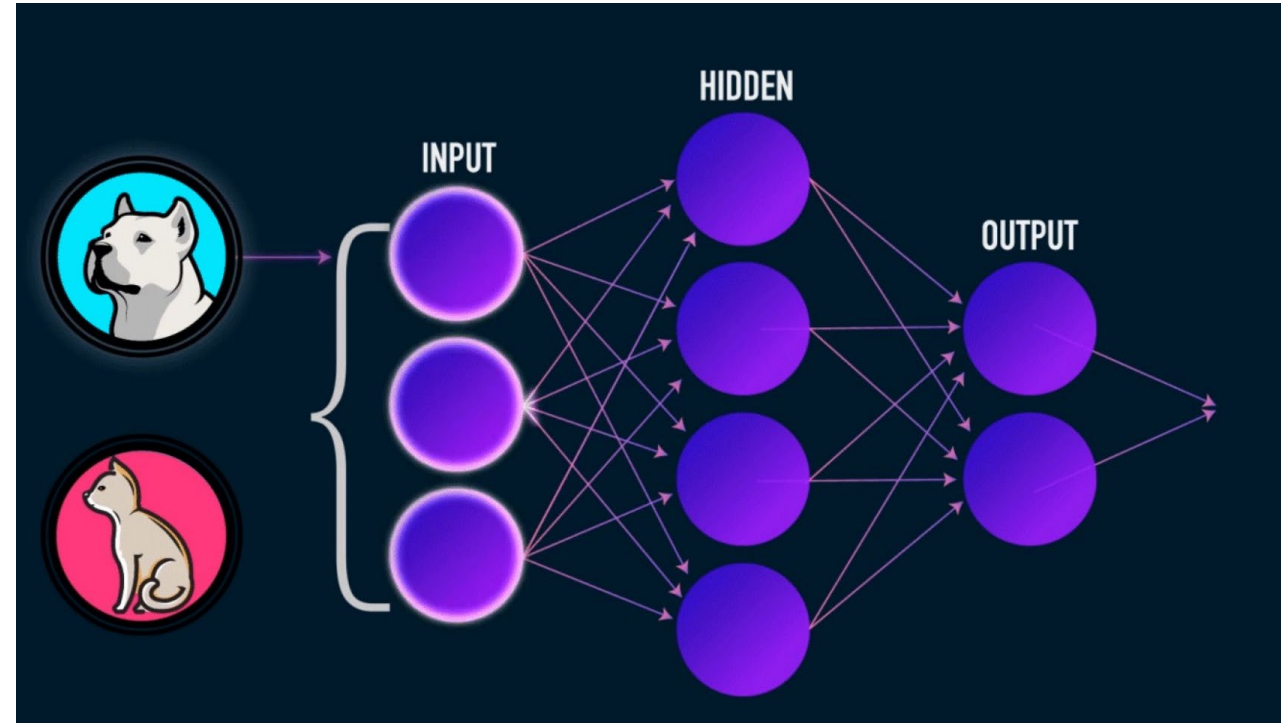
- Inventado por **Frank Rosenblatt** en 1958.
- El primer reporte sobre redes funcionales multicapas fue publicado en 1965 por Ivakhnenko y Lapa, y se conoce como el método de agrupamiento para el manejo de datos.
- Despertaron mucho interés, pero finalmente en 1969 el libro "Perceptrons" de Marvin Minsky (MIT) provocó lo que se denomina *el invierno de la IA*.
- En 1982, Paul Werbos aplicó el algoritmo de backpropagation a los MLPs de la forma en que se realiza actualmente. Pero no se disponía de Hw suficientemente potente.
- En la década de los 2000, con la popularización de las GPUs como Hw para el entrenamiento de las ANN, empezaron a utilizarse masivamente.
- En 2010 Andrew NG funda Google Brain, popularizando las redes convolucionales.

PERCEPTRON MULTICAPA (MLP)

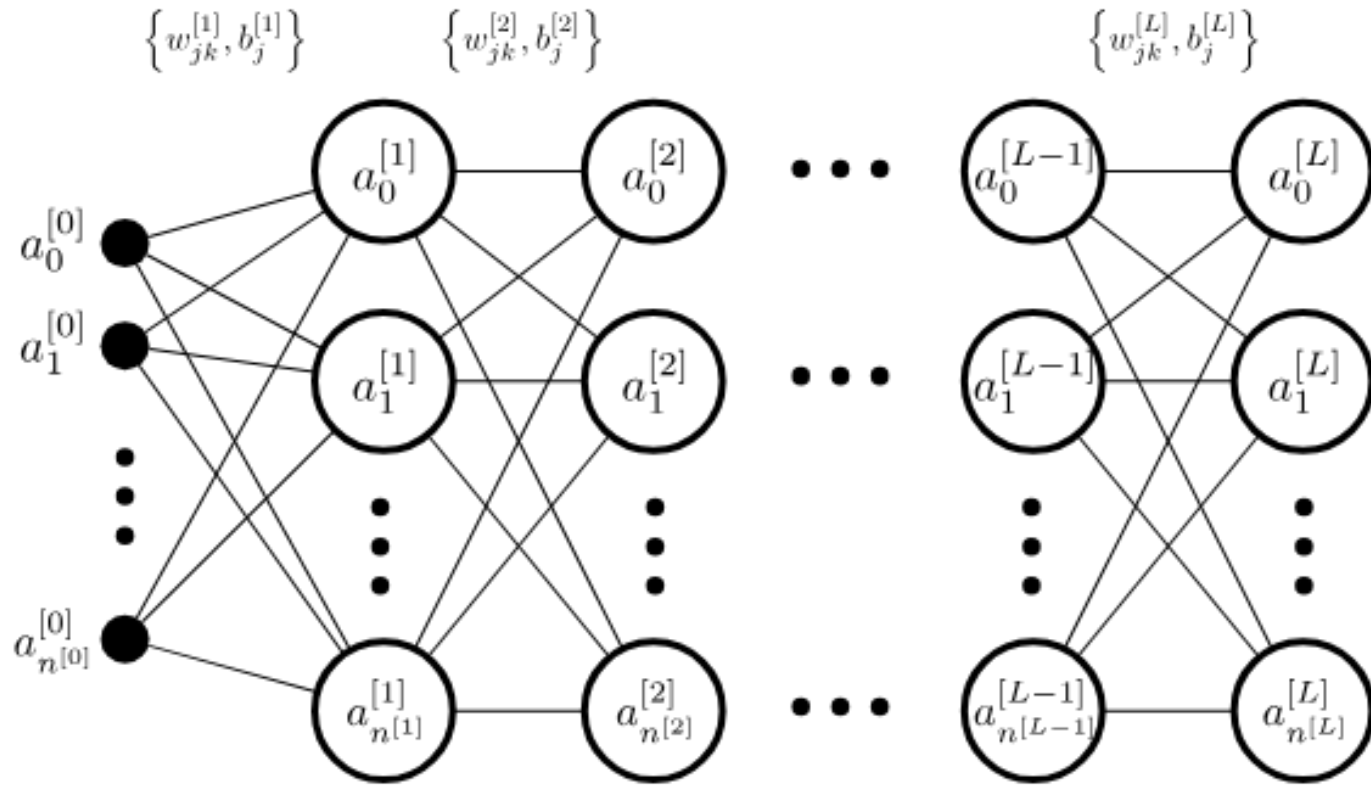
Ignacio Gómez
2025

Perceptrón Multicapa (MLP).

- Un perceptrón simple solo puede resolver problemas linealmente separables. Por ejemplo, puede clasificar datos en dos categorías cuando una línea recta puede separarlas en un espacio bidimensional. Sin embargo, no puede manejar problemas más complejos como el problema XOR, que no es linealmente separable.
- Para superar las limitaciones del perceptrón simple, se utilizan perceptrones multicapa (MLP). En un MLP, múltiples perceptrones están organizados en capas (capa de entrada, capas ocultas, y capa de salida). Esto permite a la red aprender representaciones más complejas y resolver problemas no linealmente separables.



Perceptrón Multicapa (MLP). Nomenclatura.



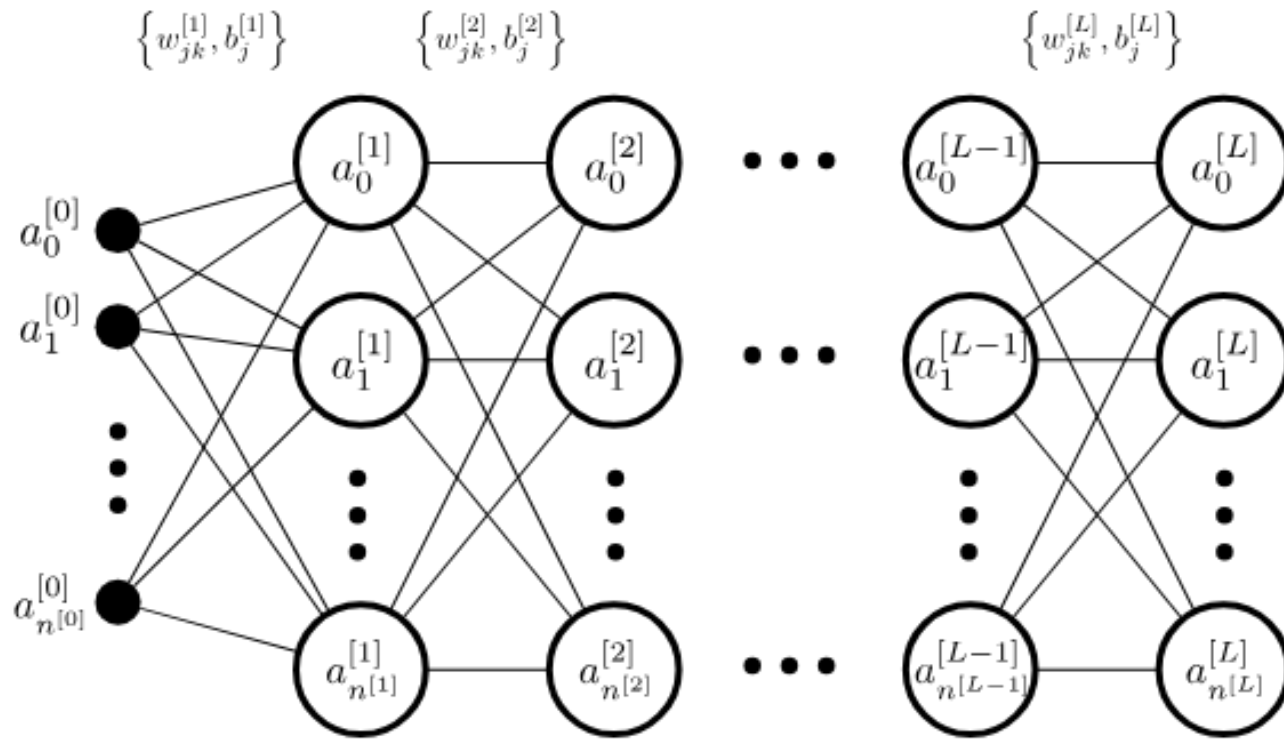
Capa de
entrada

Capas ocultas

Capa de
salida

- **Capa de entrada:** No realiza cálculos; simplemente recibe los datos de entrada. Su tamaño corresponde al número de características de entrada. No cuenta a la hora de determinar el número de capas de la red.
- **Capas ocultas:** Una o más capas entre la entrada y la salida. Cada neurona en estas capas toma una combinación ponderada de las salidas de la capa anterior y aplica una función de activación.
- **Capa de salida:** Genera la salida final de la red. Su tamaño depende del número de clases (en clasificación) o del número de valores a predecir (en regresión).

Perceptrón Multicapa (MLP). Forward Propagation.



$$z_j^{[l](i)} = \sum_{k=1}^{n^{[l-1]}} w_{jk}^{[l]} a_k^{[l-1](i)} + b_j^{[l]}$$

$$a_j^{[l](i)} = g^{[l]} \left(z_j^{[l](i)} \right)$$

$$Z^{[l]} = W^{[l]} A^{[l-1]} + B^{[l]}$$

$$A^{[l]} = g^{[l]} \left(Z^{[l]} \right)$$

$$A^{[l]}, Z^{[l]} \in \mathbb{R}^{(n^{[l]} \times m)}$$

$$W^{[l]} \in \mathbb{R}^{(n^{[l]} \times n^{[l-1]})}$$

$$B^{[l]} \in \mathbb{R}^{(n^{[l]} \times m)}$$

Perceptrón Multicapa (MLP). Ejercicio

Dada una red neuronal de **3 capas** (1 de entrada (no cuenta), **1 de salida y 2 ocultas**) con la siguiente arquitectura:

- La capa de entrada tiene 4 entradas $X = \begin{cases} x_1 \\ x_2 \\ x_3 \\ x_4 \end{cases}$
- Tenemos 10 casos
- La primera capa oculta tiene 3 nodos de activación.
- La segunda capa oculta tiene 4 nodos de activación.
- La capa de salida tiene 2 nodos.

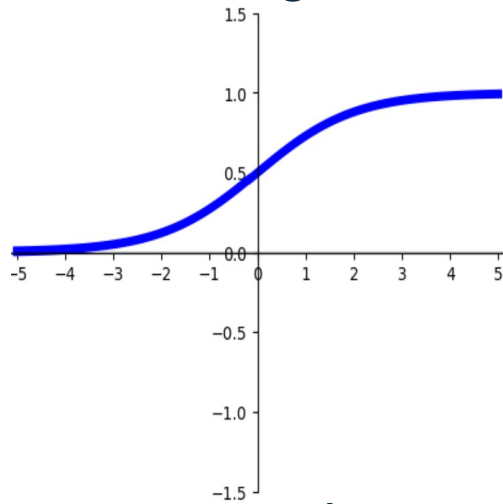
Determina la dimensión de las matrices de peso entre cada capa y de los tensores del proceso

Perceptrón Multicapa (MLP). Ejercicio

Capa	Componente	Símbolo	Dimensión (filas × columnas)
Entrada	Matriz de entrada	X (o $A^{[0]}$)	4×10
Capa 1 (Oculta)	Matriz de pesos	$W^{[1]}$	3×4
	Combinación lineal	$Z^{[1]}$	3×10
	Vector de sesgo	$b^{[1]}$	3×1 (se expande a 3×10)
Capa 2 (Oculta)	Matriz de pesos	$W^{[2]}$	4×3
	Combinación lineal	$Z^{[2]}$	4×10
	Vector de sesgo	$b^{[2]}$	4×1 (se expande a 4×10)
Capa 3 (Salida)	Matriz de pesos	$W^{[3]}$	2×4
	Combinación lineal	$Z^{[3]}$	2×10
	Vector de sesgo	$b^{[3]}$	2×1 (se expande a 2×10)
Salida	Predicción final	$\hat{Y}$ (o $A^{[3]}$)	2×10

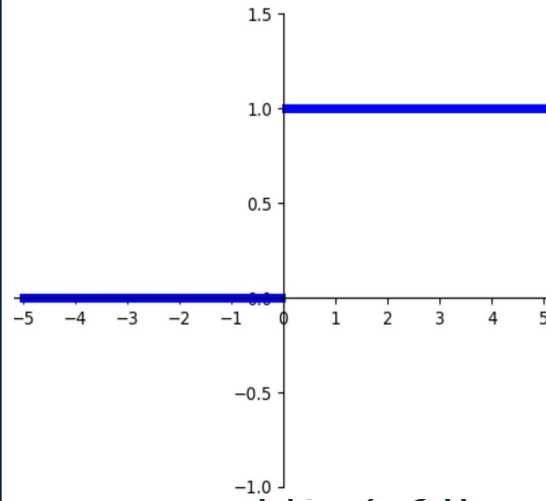
Funciones de activación.

Función Sigmoide



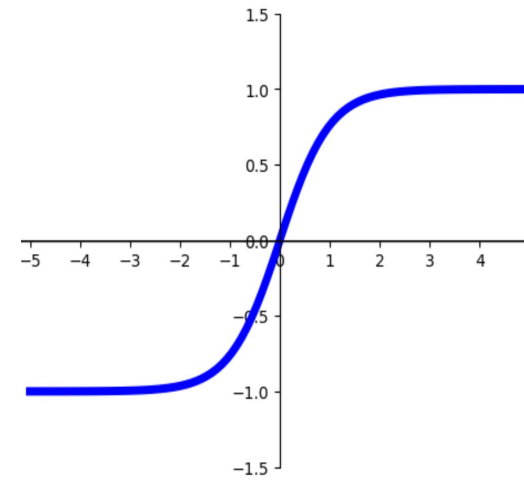
$$g(z) = \frac{1}{1 + e^{-z}}$$

Función Escalón



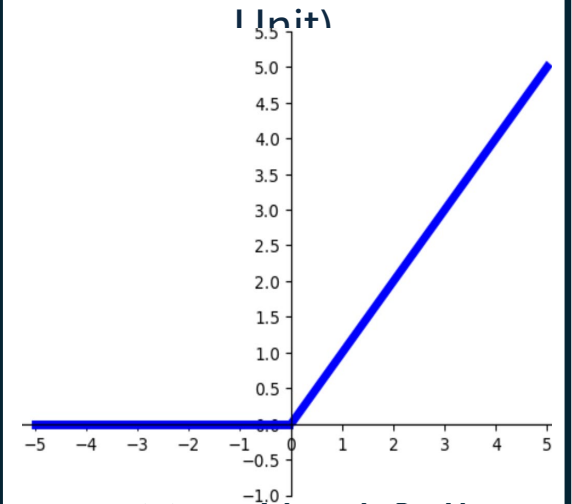
$$g(z) = \begin{cases} 1; & z \geq 0 \\ 0; & z < 0 \end{cases}$$

Tangente Hiperbólica



$$g(z) = \tanh(z)$$

ReLU (Rectified Linear



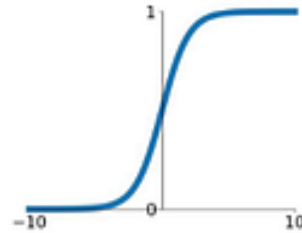
$$g(z) = \begin{cases} z; & z \geq 0 \\ 0; & z < 0 \end{cases}$$

Funciones de activación

Activation Functions

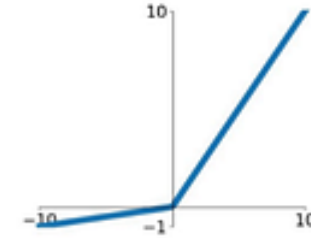
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



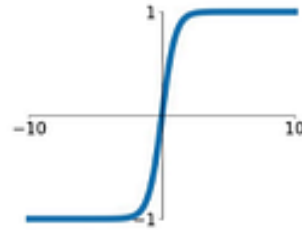
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

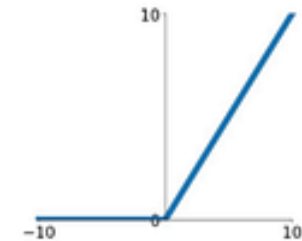


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

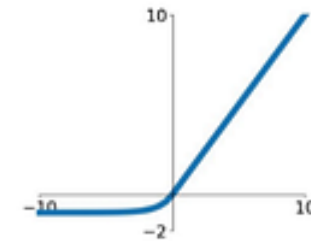
ReLU

$$\max(0, x)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Perceptrón Multicapa (MLP). Clasificación binaria

Objetivo: Clasificar un ejemplo en una de dos categorías mutuamente excluyentes (ej. Spam o No Spam). Requiere un solo nodo de salida.

1. Activación de la Capa de Salida:

- **Función Sigmoide:** Aplasta la salida para que represente una probabilidad.
- $\sigma(z) = \frac{1}{1+e^{-z}}$
- Rango de salida: $(0, 1)$

2. Función de Coste (Pérdida):

- **Entropía Cruzada Binaria (Binary Cross-Entropy):**
-

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

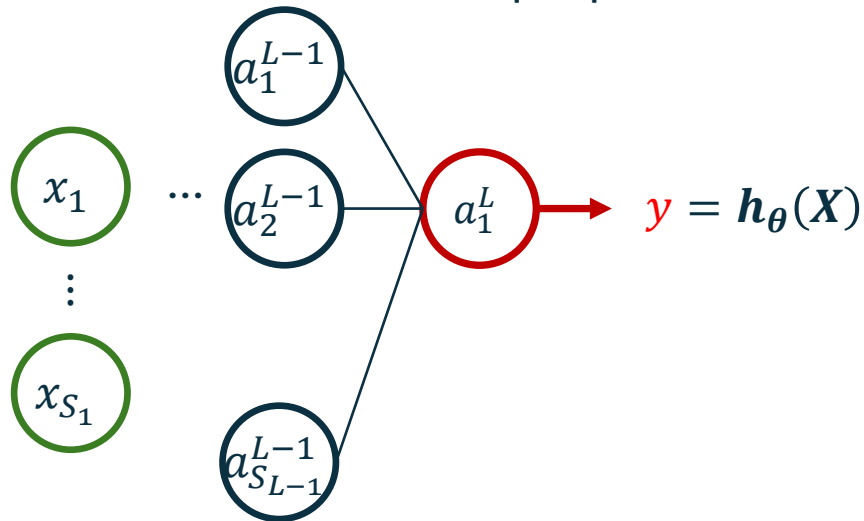
Clasificación binaria

Función de coste: Lo primero que tenemos que hacer es definir la función de error – coste que vamos a utilizar.

Va a depender del problema que estemos resolviendo.

- Si se trata de un problema de clasificación binaria lo habitual es usar la **Función De Entropía Cruzada** (Cross-Entropy)

Solo una neurona en la salida
que puede tomar valores 0/1



$$E(\theta; Y, \bar{Y}) = J(\theta) =$$

$$= -\frac{1}{M} \sum_{m=1}^M [\bar{y}^m \ln(h_{\theta}(X^m)) + (1 - \bar{y}^m) \ln(1 - h_{\theta}(X^m))]$$

Perceptrón Multicapa (MLP). Clasificación multiclase

Objetivo: Clasificar un ejemplo en una de K categorías mutuamente excluyentes ($K > 2$). Requiere K nodos de salida.

1. Activación de la Capa de Salida:

- **Función Softmax:** Convierte un vector de valores reales en una distribución de probabilidad (la suma de todas las salidas es 1).
- $\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$

2. Función de Coste (Pérdida):

- **Entropía Cruzada Categórica (Categorical Cross-Entropy):**

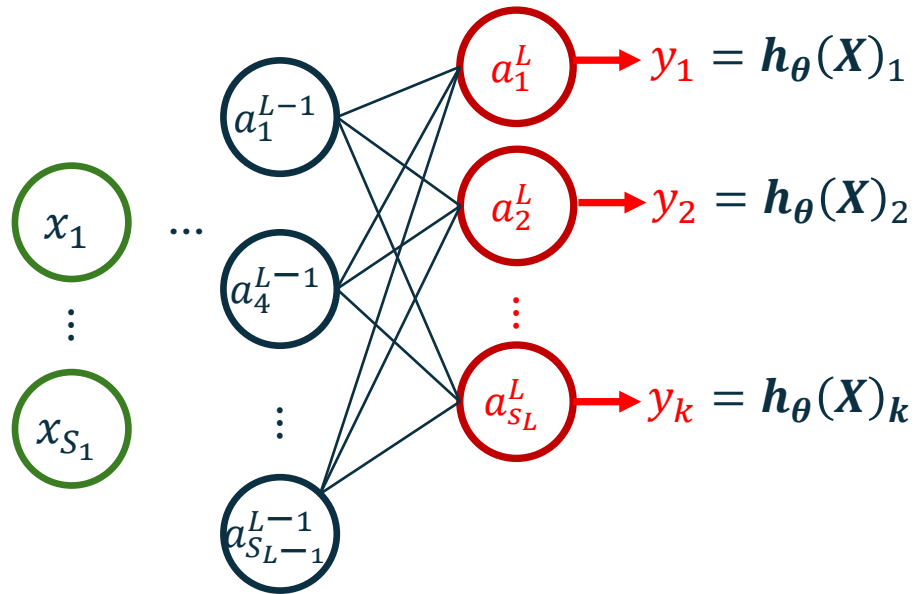
-

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{i,k} \log(\hat{y}_{i,k})$$

- *Nota:* $y_{i,k}$ suele estar codificado en *One-Hot*.

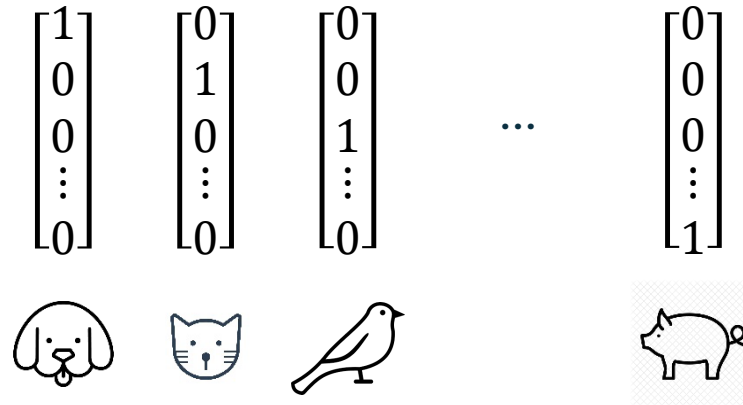
Backpropagation. Función de coste

- Si se trata de un problema de clasificación múltiple usamos la **Función De Entropía Cruzada** (Cross-Entropy) en cada clase.



Una neurona por clase en la salida que puede tomar valores 0/1.

La función de activación de la última capa suele ser una softmax exponencial que asigna un 1 a la clase con mayor probabilidad.



$$E(\theta; Y, \bar{Y}) = J(\theta) = -\frac{1}{M} \sum_{m=1}^M \sum_{k=1}^K \left[\bar{y}_k^m \ln(h_{\theta}(X^m)_k) + (1 - \bar{y}_k^m) \ln(1 - h_{\theta}(X^m)_k) \right]$$

Perceptrón Multicapa (MLP). Problema de regresión

Objetivo: Predecir un valor numérico continuo (ej. precio de una casa, temperatura).

1. Activación de la Capa de Salida:

- **Función Lineal (Identidad):** La salida de la red no se transforma.
- $f(z) = z$
- Rango de salida: $(-\infty, \infty)$

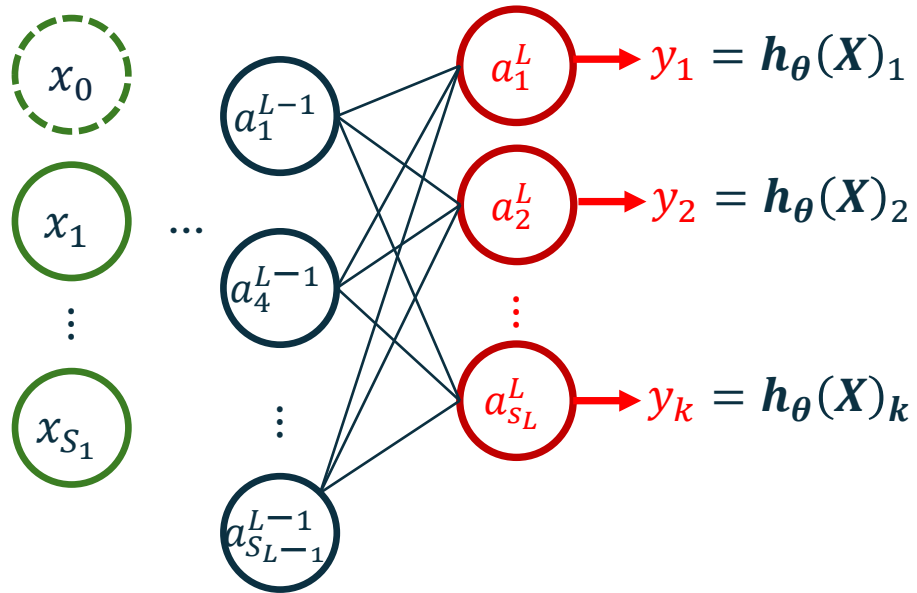
2. Función de Coste (Pérdida):

- **Error Cuadrático Medio (MSE - Mean Squared Error):** Penaliza fuertemente los errores grandes.
-

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Backpropagation. Función de coste

- Si se trata de un problema de regresión se usa el **Error Cuadrático Medio** (MSE) o el **Error Absoluto Medio** (MAE).



Una neurona por dimensión de del espacio de salida. Cada neurona devuelve el valor de la coordenada k-ésima de $\mathbf{h}_\theta(\mathbf{X})$

MSE:

$$E(\theta; Y, \bar{Y}) = J(\theta) = -\frac{1}{M} \sum_{m=1}^M \|\bar{Y}^m - \mathbf{h}_\theta(\mathbf{X})^m\|^2$$

MAE:

$$E(\theta; Y, \bar{Y}) = J(\theta) = -\frac{1}{M} \sum_{m=1}^M \|\bar{Y}^m - \mathbf{h}_\theta(\mathbf{X})^m\|_1$$

Si el problema tiene valores atípicos que pueden distorsionar los resultados es conveniente usar el MAE.

Perceptrón Multicapa (MLP). Funciones de coste y activaciones

Tipo de Problema	Activación (Salida)	Función de Coste	Descripción y Caso de Uso
Clasificación Binaria	Sigmoide	Entropía Cruzada Binaria (Binary Cross-Entropy)	Objetivo: Predecir entre dos clases (0 o 1). Ejemplo: Detectar si un correo es Spam o No Spam.
Clasificación Múltiple (Multi-clase)	Softmax	Entropía Cruzada Categórica (Categorical Cross-Entropy)	Objetivo: Clasificar en <i>una sola</i> clase dentro de múltiples opciones excluyentes. Ejemplo: Reconocer perro, gato o pájaro.
Clasificación Multietiqueta	Sigmoide	Entropía Cruzada Binaria (Binary Cross-Entropy)	Objetivo: Asignar múltiples clases simultáneas a una sola entrada. Ejemplo: Película de "Acción" y "Comedia".
Regresión	Lineal (Identidad)	Error Cuadrático Medio (MSE), Error Absoluto Medio (MAE)	Objetivo: Predecir un valor numérico continuo. Ejemplo: Predecir el precio de una casa.

Backpropagation. Función de coste + Regularización

- Tanto en los problemas de Regresión como en los de Clasificación se puede añadir un término a la función de pérdida. Lo veremos más tarde.

B-Clas: $J(\theta) = -\frac{1}{M} \sum_{m=1}^M [\bar{y}^m \ln(h_{\theta}(X^m)) + (1 - \bar{y}^m) \ln(1 - h_{\theta}(X^m))] + \frac{\lambda}{2M} \sum_{i,j,l} (\theta_{ij}^l)^2$

M-Clas: $J(\theta) = -\frac{1}{M} \sum_{m=1}^M \sum_{k=1}^K [\bar{y}_k^m \ln(h_{\theta}(X^m))_k + (1 - \bar{y}_k^m) \ln(1 - h_{\theta}(X^m)_k)] + \frac{\lambda}{2M} \sum_{i,j,l} (\theta_{ij}^l)^2$

MSE: $J(\theta) = -\frac{1}{M} \sum_{m=1}^M \|\bar{Y}^m - h_{\theta}(X)^m\|^2 + \frac{\lambda}{2M} \sum_{i,j,l} (\theta_{ij}^l)^2$

MAE: $J(\theta) = -\frac{1}{M} \sum_{m=1}^M \|\bar{Y}^m - h_{\theta}(X)^m\|_1 + \frac{\lambda}{2M} \sum_{i,j,l} (\theta_{ij}^l)^2$

BACKPROPAGATION

Ignacio Gómez
2025

Backpropagation.

Queremos hallar los valores de los parámetros que minimizan la función de pérdida. Para eso calculamos las derivadas de la función de coste respecto a los parámetros y las utilizamos para actualizar los pesos según el algoritmo del gradiente:

$$\begin{aligned}W^{[l]} &= W^{[l]} - \alpha dW^{[l]} \\ B^{[l]} &= B^{[l]} - \alpha dB^{[l]}\end{aligned}$$

Backpropagation.

Aplicando la regla de la cadena, vamos calculando las derivadas capa a capa:

$$z_j^{[l](i)} = \sum_{k=1}^{n^{[l-1]}} w_{jk}^{[l]} a_k^{[l-1](i)} + b_j^{[l]}$$

$$a_j^{[l](i)} = g^{[l]} \left(z_j^{[l](i)} \right)$$

$$da_j^{[l](i)} = \frac{\partial \mathcal{L}}{\partial a_j^{[l](i)}} = \sum_{j'=1}^{n^{[l+1]}} \frac{\partial \mathcal{L}}{\partial z_{j'}^{[l+1](i)}} \frac{\partial z_{j'}^{[l+1](i)}}{\partial a_j^{[l](i)}}$$

$$= \sum_{j'=1}^{n^{[l+1]}} dz_{j'}^{[l+1](i)} w_{j'j}^{[l+1]}$$

$$dA^{[l]} = W^{[l+1]\top} dZ^{[l+1]}$$

$$dz_j^{[l](i)} = \frac{\partial \mathcal{L}}{\partial z_j^{[l](i)}} = \sum_{j'=1}^{n^{[l]}} \frac{\partial \mathcal{L}}{\partial a_{j'}^{[l](i)}} \frac{\partial a_{j'}^{[l](i)}}{\partial z_j^{[l](i)}}$$

$$= \sum_{j'=1}^{n^{[l]}} da_{j'}^{[l](i)} g^{[l]'} \left(z_j^{[l](i)} \right) \delta_{j'j}$$

$$dZ^{[l]} = dA^{[l]} \odot g^{[l]'} \left(Z^{[l]} \right)$$

Backpropagation.

Para la función de coste total de todo el dataset:

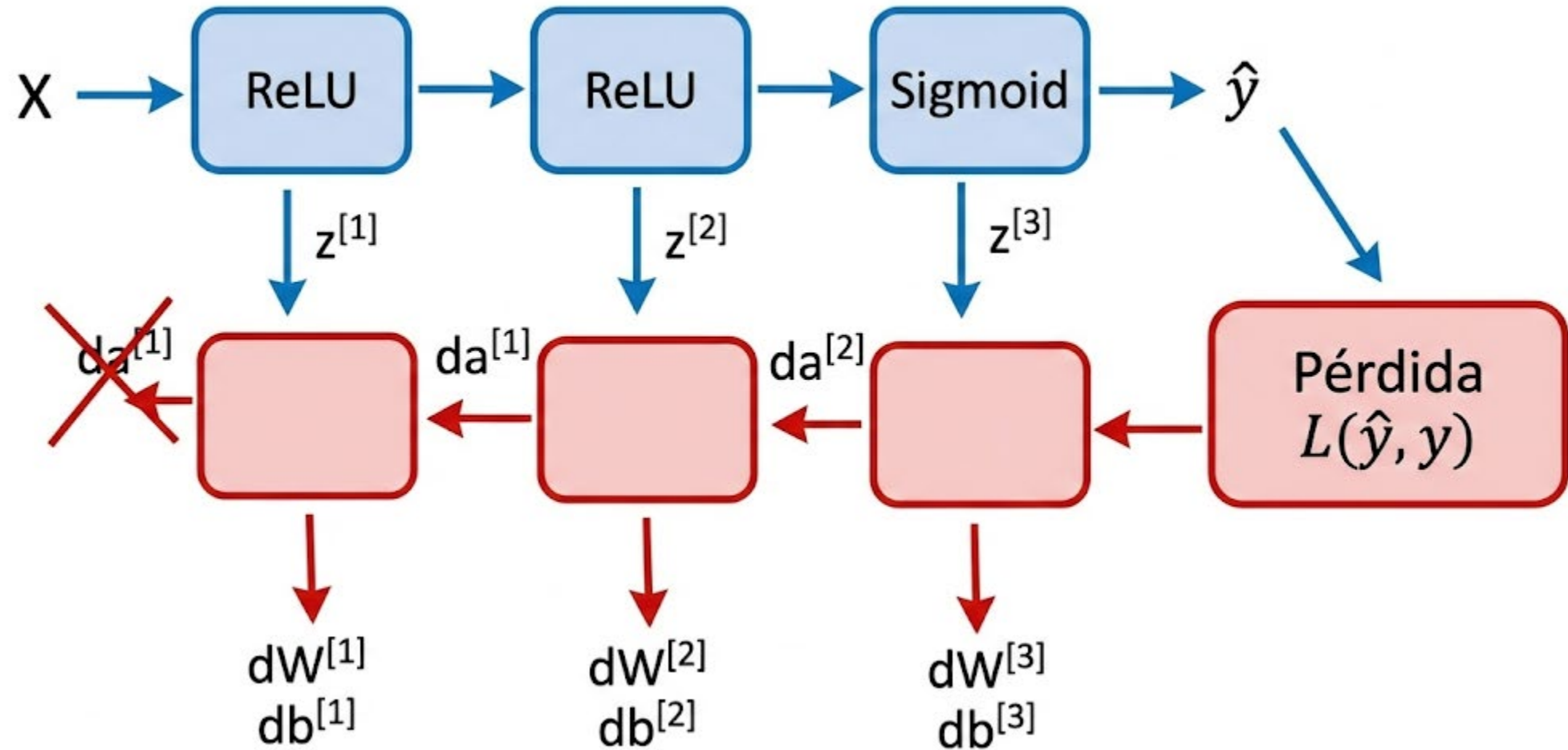
$$\begin{aligned} dw_{jk}^{[l]} &= \frac{\partial \mathcal{C}}{\partial w_{jk}^{[l]}} = \sum_{i=1}^m \sum_{j'=1}^{n^{[l]}} \frac{\partial \mathcal{C}}{\partial z_{j'}^{[l](i)}} \frac{\partial z_{j'}^{[l](i)}}{\partial w_{jk}^{[l]}} \\ &= \frac{1}{m} \sum_{i=1}^m dz_j^{[l](i)} a_k^{[l-1](i)} \end{aligned}$$

$$dW^{[l]} = \frac{1}{m} dZ^{[l]} A^{[l-1]\top}$$

$$\begin{aligned} db_j^{[l]} &= \frac{\partial \mathcal{C}}{\partial b_j^{[l]}} = \sum_{i=1}^m \sum_{j'=1}^{n^{[l]}} \frac{\partial \mathcal{C}}{\partial z_{j'}^{[l](i)}} \frac{\partial z_{j'}^{[l](i)}}{\partial b_j^{[l]}} \\ &= \frac{1}{m} \sum_{i=1}^m dz_j^{[l](i)} \end{aligned}$$

$$dB^{[l]} = \frac{1}{m} \sum_{i=1}^m \left(dz^{[l]} \right)_{ji}$$

Backpropagation.



Inicialización de los parámetros

- Inicializar los parámetros en cero es mala idea, hace que los gradientes sean iguales y que los parámetros de la capa sean siempre idénticos, así que todas las neuronas de la capa calculan exactamente lo mismo.
- Lo más típico es la inicialización Random. Interesa que los valores sean pequeños para permanecer en la zona central de las sigmoides donde las derivadas son más grandes. Por ejemplo:

```
np.random.randn(n, m) * 0.01
```


Bibliografia:

<https://dennybritz.com/posts/wildml/implementing-a-neural-network-from-scratch/>

Cuadernos:

1. Tutorial_Introduccion_Redes_Neuronales_Desde_Cero.ipynb
2. Tutorial_Introduccion_Redes_Neuronales_Desde_Cero regresion.ipynb
3. Tutorial_Introduccion_Redes_Neuronales_Desde_Cero Multiple.ipynb